TabTune: A Unified Library for Inference and Fine-Tuning Tabular Foundation Models

Aditya Tanna
Pratinav Seth
aditya.tanna@lexsi.ai
pratinav.seth@lexsi.ai
Lexsi Labs
Mumbai, India

Mohamed Bouadi
mohamed.bouadi@lexsi.ai
Lexsi Labs
Paris, France

Vinay Kumar Sankarapu
v.k@lexsi.ai
Lexsi Labs
London, United Kingdom

Abstract

Tabular foundation models (TFMs) have become powerful solutions for structured data problems, but their inconsistent APIs and preprocessing steps remain major obstacles for practitioners. We introduce TabTune, a unified library that provides a simple interface for multiple leading tabular foundation models. TabTune[1] reduces fragmentation across existing tools by standardizing preprocessing, supporting inference and multiple fine-tuning modes, offering a broad suite of evaluation metrics, and enabling benchmarking through a single API. Our interactive Web-based demo,[2] [3], highlights three primary use cases: (1) quick-start examples for running models with minimal code, (2) comparative benchmarking, and (3) advanced responsible evaluation using fairness and calibration metrics. Target applications such as churn prediction, fraud detection, and behavioral modeling illustrate how TabTune streamlines the practical adoption of TFMs in large-scale Web systems.

CCS Concepts

• **Computing methodologies** → *Machine learning algorithms*; *Learning paradigms*; • **Information systems** → Web applications.

Keywords

tabular foundation models, fine-tuning, web applications

ACM Reference Format:
Aditya Tanna, Pratinav Seth, Mohamed Bouadi, and Vinay Kumar Sankarapu. 2026. TabTune: A Unified Library for Inference and Fine-Tuning Tabular Foundation Models. In *Companion Proceedings of the ACM Web Conference 2026 (WWW Companion '26), April 13–17, 2026, Dubai, United Arab Emirates.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3774905.3793137

1 Introduction

The Web ecosystem continually generates vast amounts of *tabular* data from user interactions, transaction logs, ad impressions, and platform telemetry. Traditionally, such data has been handled with task-specific models trained from scratch using gradient boosting or shallow neural networks. Recently, however, *tabular foundation models* (TFMs) such as TabPFN [5], TabICL [8], ORION [1, 2] and others have emerged as pre-trained architectures that can be quickly adapted to new tabular tasks in a zero- or few-shot manner. They promise stronger performance in low-data regimes, faster iteration cycles, and a standardized modeling layer across heterogeneous Web applications. For modern Web platforms, this shift is particularly attractive: a single TFM can support high-impact tasks such as predicting subscription churn, assessing lending risk in online financial platforms, detecting fraudulent e-commerce transactions, and optimizing click-through-rate in Web advertising. In all of these settings, decisions affect users at scale and are subject to regulatory and ethical scrutiny. TFMs therefore offer an appealing path toward reusable, high-capacity models.

In practice, however, the TFM ecosystem is highly fragmented. Key challenges include: (1) **API inconsistency**: each model (TabPFN, TabICL, etc.) exposes a different interface, complicating comparison and production integration; (2) **Preprocessing complexity**: bespoke input formats and feature transformations demand deep familiarity with each implementation; (3) **Evaluation gaps**: limited tooling for systematic assessment beyond accuracy, especially for calibration and fairness metrics critical to Web-scale decision systems; and (4) **Deployment barriers**: no unified framework for serving, fine-tuning, and reusing checkpoints across applications.

TabTune addresses these issues by providing, to the best of our knowledge, the first unified library explicitly designed around tabular foundation models. Its main contributions are: (1) a simple, consistent API shared across all supported TFMs; (2) model-aware preprocessing that automatically applies the transformations for a TFM; (3) multiple fine-tuning strategies, including supervised fine-tuning (SFT), meta-learning–style fine-tuning (MLFT), and parameter-efficient fine-tuning (PEFT); (4) a comprehensive evaluation layer with calibration and group-fairness metrics for responsible Web deployment; and (5) a benchmarking module that compares TFMs on a given dataset to help practitioners select and deploy the most appropriate model.

By unifying preprocessing, adaptation, and deployment-oriented evaluation (including calibration and fairness) for tabular *foundation models* within a single API, TabTune enables Web companies to use TFMs as a reusable modeling layer across products and decision workflows.

2 System Overview

TabTune integrates multiple state-of-the-art tabular foundation models through four core components (Figure 1): TabularPipeline,

[1]Code: https://github.com/Lexsi-Labs/TabTune
[2]Demo: https://lightning.ai/pratinavsetharya/environments/tabtune-demo
[3]Video: https://drive.google.com/drive/folders/1b3G9zDP7drMch7pQOxRZELwAHY00lFfh

This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW Companion '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2308-7/2026/04
https://doi.org/10.1145/3774905.3793137

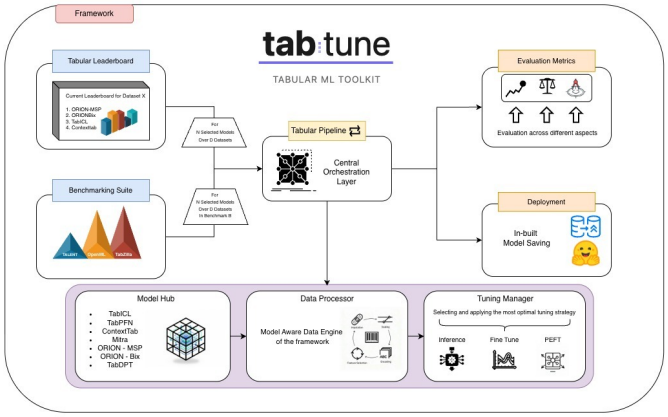


Figure 1: TabTune System Architecture: The library integrates multiple tabular foundation models through four core components (TabularPipeline, DataProcessor, TuningManager, and TabularLeaderboard).

DataProcessor, TuningManager, and TabularLeaderboard. **Core Architecture :**

- **TabularPipeline** serves as the main user-facing API, providing scikit-learn compatible methods (.fit(), .predict(), .predict_proba()) that work consistently across all supported models. The pipeline handles model initialization, data flow, and result processing.
- **DataProcessor** implements model-aware preprocessing, automatically selecting the appropriate transformations based on the chosen foundation model. Each model has a specialized preprocessor (e.g., TabPFNPreprocessor, TabICLPreprocessor) that handles categorical encoding, numerical scaling, and feature engineering specific to that model's requirements.
- **TuningManager** orchestrates the model adaptation process, supporting three strategies: (1) *inference* for zero-shot predictions, (2) *base-ft* for full parameter fine-tuning, and (3) *peft* for parameter-efficient fine-tuning using LoRA adapters. [6]
- **TabularLeaderboard** enables systematic comparison of multiple models and strategies on the same dataset, providing ranked results and performance metrics.

Supported Models : TabTune currently supports seven state-of-the-art tabular foundation models: TabPFN [5], TabICL [8], OrionMSP [2], OrionBiX [1], Mitra [10], ContextTab [9], and TabDPT [7]. Each model is wrapped with a consistent interface while preserving its unique capabilities and preprocessing requirements.

3 Basic Usage

TabTune provides a uniform API that works across all supported tabular foundation models. Users configure a model and adaptation strategy once and can then call familiar methods such as `fit()`, `predict()`, and `predict_proba()` for a wide range of Web tasks. This keeps the public API stable even as new TFM checkpoints are released and allows practitioners to reuse existing infrastructure for future models.

3.1 Inference Mode

For quick predictions without additional training, TabTune offers a pure inference (zero-shot) mode that evaluates pretrained TFMs directly on the target dataset:

```
from tabtune import TabularPipeline
pipeline = TabularPipeline(
    model_name="TabPFN",
    tuning_strategy="inference")
pipeline.fit(X_train, y_train)
predictions = pipeline.predict(X_test)
```

3.2 Fine-Tuning Modes

Beyond zero-shot use, TabTune exposes several adaptation strategies through simple configuration flags. Each of these paradigms is configured by changing a few arguments, without rewriting training loops. The examples below illustrate all four combinations used in our experiments and demo.

Supervised fine-tuning (SFT) performs full-parameter optimization (AdamW, typically up to 10 epochs with early stopping), making it suitable for medium-sized datasets.

```
from tabtune import TabularPipeline
sft_pipeline = TabularPipeline(
    model_name="TabPFN",
    tuning_strategy="base-ft",
    finetune_mode="sft",
    tuning_params={"epochs": 3, "learning_rate": 3e-5})
sft_pipeline.fit(X_train, y_train)
```

Meta-learning fine-tuning uses episodic training with small support sets (e.g., 48–512 samples) to better align the TFM with the target task distribution.

```
from tabtune import TabularPipeline
mlft_pipeline = TabularPipeline(
    model_name="OrionMSP",
    tuning_strategy="base-ft",
    finetune_mode="meta-learning",
    tuning_params={"epochs": 3, "learning_rate": 2e-5})
mlft_pipeline.fit(X_train, y_train)
```

Parameter-efficient fine-tuning (PEFT) applies LoRA adapters (e.g., rank r=8, α=16), reducing memory usage; PEFT can be combined with both SFT and meta-learning variants.

```
from tabtune import TabularPipeline
peft_sft_pipeline = TabularPipeline(
    model_name="TabDPT",
    tuning_strategy="peft",
    finetune_mode="sft",
    tuning_params={"epochs": 5, "learning_rate": 1e-4})
peft_sft_pipeline.fit(X_train, y_train)

peft_ml_pipeline = TabularPipeline(
    model_name="TabICL",
    tuning_strategy="peft",
    finetune_mode="meta-learning",
    tuning_params={"epochs": 5, "learning_rate": 1e-4})
peft_ml_pipeline.fit(X_train, y_train)
```

3.3 Evaluation

TabTune provides built-in evaluation for performance, calibration, and fairness, as well as multi-model benchmarking. Performance is measured using Accuracy, weighted F1, and more. Calibration is assessed using Expected Calibration Error (ECE), Maximum Calibration Error (MCE) [4], and Brier Score [3]. Fairness is evaluated

via Statistical Parity Difference (SPD), Equalized Odds Difference (EOD), and Equal Opportunity Difference (EOpD). These metrics are computed with a single call and surfaced in the demo interface.

```
from tabtune import TabularPipeline
# Evaluation for a single model
pipeline = TabularPipeline(
    model_name="TabICL",
    tuning_strategy="inference")
pipeline.fit(X_train, y_train)

performance = pipeline.evaluate(X_test, y_test)
calibration = pipeline.evaluate_calibration(
    X_test, y_test, n_bins=15)
fairness = pipeline.evaluate_fairness(
    X_test, y_test, sensitive_features="gender")
```

3.4 Benchmarking

The `TabularLeaderboard` component streamlines multi-model and multi-strategy comparison for multiple TFMs on a given dataset split, computing metrics, and returning a ranked table (e.g., by ROC_AUC).

```
from tabtune import TabularPipeline, TabularLeaderboard
# Multi-model leaderboard
leaderboard = TabularLeaderboard(
    X_train, X_test, y_train, y_test,
)
leaderboard.add_model("OrionMSP", "inference")
leaderboard.add_model("OrionBiX", "inference")
leaderboard.add_model(
    "TabPFN", "base-ft", "sft",
    tuning_params={"learning_rate": 1e-3},
)
leaderboard.add_model(
    "TabICL", "base-ft", "meta-learning",
    tuning_params={"epochs": 5},
)
leaderboard.add_model("TabDPT", "peft", "sft")
leaderboard.add_model("Mitra", "peft", "meta-learning")

results = leaderboard.run(rank_by="roc_auc_score")
```

4 Demo Illustrations

4.1 Demo Setup

The TabTune demo is hosted on Lightning Studio and runs on both free-tier CPU and GPU runtimes. Attendees interact with the system via a simple workflow:

(1) **Select or Upload Dataset:** Choose a Web-style task (e.g., churn, fraud, CTR) from the built-in collection, or upload a custom tabular dataset (CSV).
(2) **Choose Foundation Model:** Select one of the seven supported tabular foundation models.
(3) **Run Inference or Fine-Tuning:** Perform zero-shot inference or apply fine-tuning strategies (SFT, meta-learning, PEFT) with a single action.
(4) **Inspect Metrics:** View performance metrics and, when needed, a detailed dashboard with calibration and fairness metrics .
(5) **Benchmark with `TabularLeaderboard`:** Compare multiple models and tuning strategies on a shared train–test split to obtain a ranked overview.

4.2 Data Ingestion and Exploratory Analysis

This part of the demo illustrates how TabTune ingests tabular data and provides built-in exploratory analysis for the credit card default dataset (OpenML ID 1597), as shown in Figures 2 and 3. Attendees can review schema information, feature types, and basic distributions before moving on to model selection and tuning.

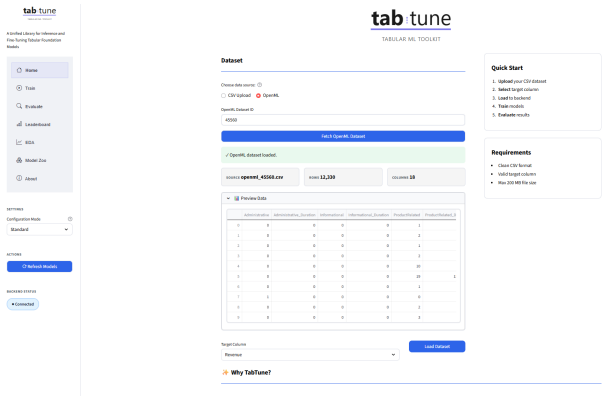


Figure 2: Interactive data ingestion and schema summary for the credit card default dataset (OpenML ID 1597).

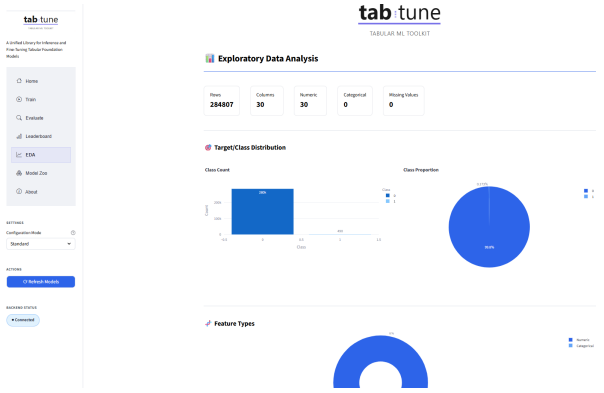


Figure 3: Exploratory data analysis (EDA) view for the credit card default dataset (OpenML ID 1597).

4.3 Fine-Tuning on a Dataset

Figure 4 illustrates the second step of the workflow, where attendees configure and run meta-learning based fine-tuning on the adult churn dataset (OpenML ID 1590). The interface summarizes performance and cost trade-offs across different fine-tuning strategies.

4.4 Advanced Evaluation: Calibration & Fairness

After a model has been trained, the demo provides an advanced evaluation view (Figure 5) for an online credit-approval setting using the COMPAS dataset. Attendees can jointly inspect predictive

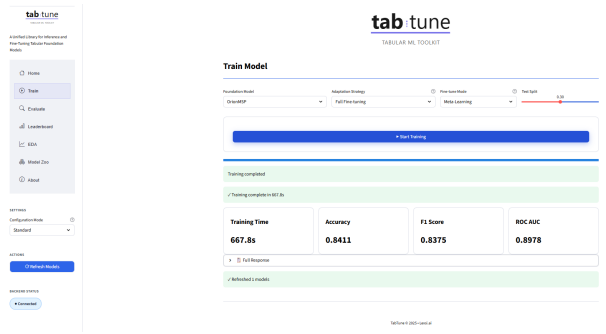


Figure 4: Meta-learning fine-tuning strategies for the adult churn dataset (OpenML ID 1590).

performance, calibration (ECE, Brier score, MCE), and group fairness metrics such as Statistical Parity Difference (SPD), Equalized Odds Difference (EOD), and Equal Opportunity Difference (EOpD) across sensitive attributes (e.g., gender, age group). Each analysis (performance, calibration, or fairness) can also be run individually, allowing users to focus on the dimensions most relevant to their deployment scenario.

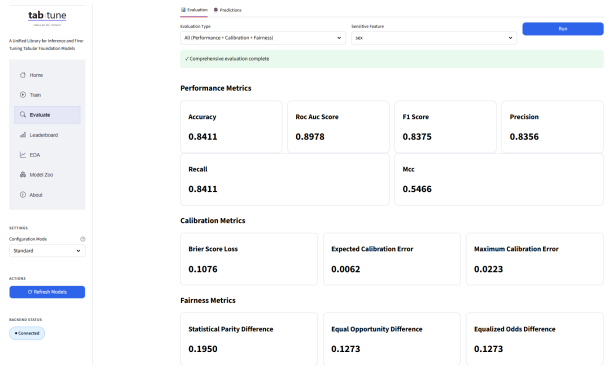


Figure 5: Calibration and fairness analysis for the COMPAS dataset, including ECE, Brier score (BS), MCE, and group fairness metrics (SPD, EOpD, EOD).

4.5 Benchmarking Multiple Models

The final demo presents TabTune's multi-model benchmarking on an online shopper intention prediction dataset (OpenML ID 45560). Using `TabularLeaderboard`, attendees can compare all seven TFMs and multiple tuning strategies side by side. The leaderboard reports ranked performance across metrics (e.g., ROC AUC, F1, mean rank), helping identify which models and strategies are best suited for the given task and data regime.

5 Conclusion

TabTune provides, to the best of our knowledge, the first unified library dedicated to tabular foundation models, addressing the fragmented landscape of heterogeneous APIs, preprocessing pipelines, and tuning recipes. By exposing a consistent abstraction over seven

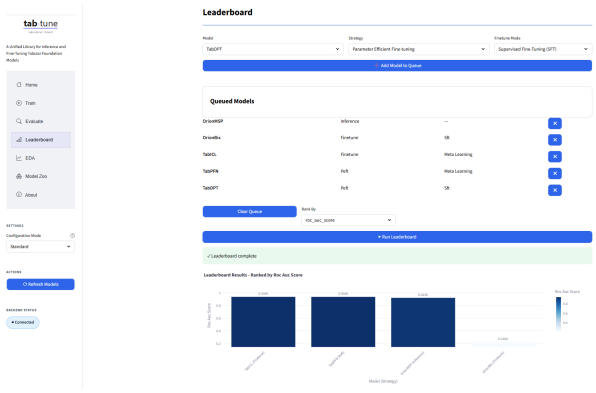


Figure 6: `TabularLeaderboard` results: ranked comparison of multiple tabular foundation models on an online shopper intention prediction task (OpenML ID 45560).

TFMs, it enables practitioners to adopt state-of-the-art tabular models for Web applications with minimal friction.

Our main contributions are: (1) a simple API for multiple tabular foundation models; (2) model-aware preprocessing; (3) support for multiple fine-tuning methods; (4) an integrated evaluation layer for performance, calibration, and fairness; and (5) a leaderboard for multi-model, multi-strategy benchmarking. Together, these components support Web-centric use cases such as customer analytics, fraud detection, recommendation, and user behavior modeling.

References

[1] Mohamed Bouadi, Pratinav Seth, Aditya Tanna, and Vinay Kumar Sankarapu. 2025. Orion-Bix: Bi-Axial Attention for Tabular In-Context Learning. arXiv:2512.00181 [cs.LG] https://arxiv.org/abs/2512.00181

[2] Mohamed Bouadi, Pratinav Seth, Aditya Tanna, and Vinay Kumar Sankarapu. 2025. Orion-MSP: Multi-Scale Sparse Attention for Tabular In-Context Learning. arXiv:2511.02818 [cs.AI] https://arxiv.org/abs/2511.02818

[3] Glenn W. Brier. 1950. Verification of Forecasts Expressed in Terms of Probability. *Monthly Weather Review* 78, 1 (1950), 1–3.

[4] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. 2017. On calibration of modern neural networks. In *International conference on machine learning*. PMLR, 1321–1330.

[5] Noah Hollmann, Samuel Müller, Katharina Eggensperger, and Frank Hutter. 2023. TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/forum?id=cp5PvcI6w8_

[6] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *Proceedings of the International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2106.09685

[7] Junwei Ma, Valentin Thomas, Rasa Hosseinzadeh, Hamidreza Kamkari, Alex Labach, Jesse C. Cresswell, Keyvan Golestan, Guangwei Yu, Maksims Volkovs, and Anthony L. Caterini. 2024. TabDPT: Scaling Tabular Foundation Models. *CoRR* abs/2410.18164 (2024). arXiv:2410.18164 doi:10.48550/ARXIV.2410.18164

[8] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. 2025. TabICL: A Tabular Foundation Model for In-Context Learning on Large Data. *CoRR* abs/2502.05564 (2025). arXiv:2502.05564 doi:10.48550/ARXIV.2502.05564

[9] Marco Spinaci, Marek Polewczyk, Maximilian Schambach, and Sam Thelin. 2025. ConTextTab: A Semantics-Aware Tabular In-Context Learner. *CoRR* abs/2506.10707 (2025). arXiv:2506.10707 doi:10.48550/ARXIV.2506.10707

[10] Xiyuan Zhang, Danielle C. Maddix, Junming Yin, Nick Erickson, Abdul Fatir Ansari, Boran Han, Shuai Zhang, Leman Akoglu, Christos Faloutsos, Michael W. Mahoney, Cuixiong Hu, Huzefa Rangwala, George Karypis, and Bernie Wang. 2025. Mitra: Mixed Synthetic Priors for Enhancing Tabular Foundation Models. arXiv:2510.21204 [cs.LG] https://arxiv.org/abs/2510.21204